

Tarefa 2 Clojure

Arthur Gama Ruiz

28 de setembro de 2026

Sumário

1	Questão 1	2
2	Questão 2	2
3	Questão 3	2
4	Questão 4	3
5	Questão 5	4
6	Questão 6	4
7	Questão 7	4
8	Questão 8	5
9	Questão 9	5
10	Questão 10	5

1 Questão 1

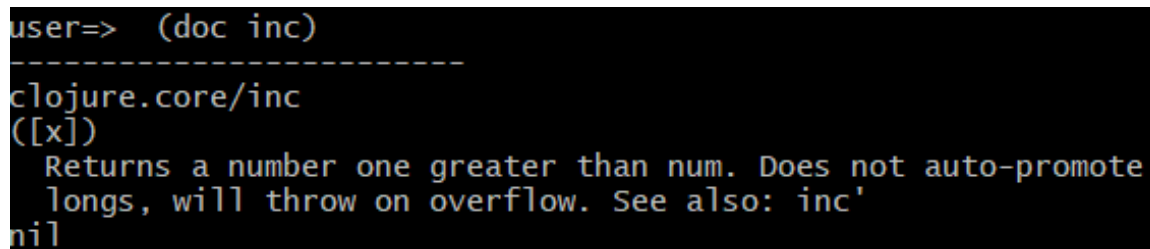
```
user=> (+ 8 5)
13
user=> (* 6 7)
42
user=> (/ 20 4)
5
user=> (- 15 9)
6
user=> (* (+ 10 5) 3)
45
```

2 Questão 2

```
user=> ( str "ProgramacaoFuncionalcom Clojure")
"ProgramacaoFuncionalcom Clojure"
user=> (str "Arthur esta aprendendo Clojure")
"Arthur esta aprendendo Clojure"
```

3 Questão 3

Funções pesquisadas: `inc` (incrementar), `lower-case` (para converter texto para minúsculo) e `map` (transforma cada item da lista).



```
user=> (doc inc)
-----
clojure.core/inc
([x])
  Returns a number one greater than num. Does not auto-promote
  longs, will throw on overflow. See also: inc'
nil
```

Figura 1: Documentação da função *inc*.

```
user=> (doc clojure.string/lower-case)
-----
clojure.string/lower-case
([s])
  Converts string to all lower-case.
```

Figura 2: Documentação da função *clojure.string/lower-case*.

```
user=> (doc map)
-----
clojure.core/map
([f] [f coll] [f c1 c2] [f c1 c2 c3] [f c1 c2 c3 & colls])
  Returns a lazy sequence consisting of the result of applying f to
  the set of first items of each coll, followed by applying f to the
  set of second items in each coll, until any one of the colls is
  exhausted. Any remaining items in other colls are ignored. Function
  f should accept number-of-colls arguments. Returns a transducer when
  no collection is provided.
```

Figura 3: Documentação da função *map*.

4 Questão 4

```
user=> (first (list 10 20 30 40))
10
user=> (last (list 10 20 30 40))
40
user=> (rest (list 10 20 30 40))
(20 30 40)
user=> (count (list 10 20 30 40))
4
```

Figura 4: Manipulação de listas.

5 Questão 5

A função *quote* diz ao compilador para não executar o que está dentro dos parênteses, mas sim apenas para ler. Poderíamos também utilizar o símbolo de apóstrofo (*'*).

```
user=> (quote ( 1 2 3))  
(1 2 3)
```

```
user=> '( + 2 3)  
(+ 2 3)
```

No segundo exemplo, símbolo *'+'* é considerado como um dado, e não a operação de adição.

6 Questão 6

```
user=> (if true "Aprovado" "Repreovado")  
"Aprovado"
```

```
user=> (if false "Aprovado" "Repreovado")  
"Repreovado"
```

Alteramos a condição para verificar o funcionamento completo da condicional *if*.

```
user=> (if false "True"(+ 100 50))  
150
```

A estrutura condicional acima retorna *True* caso a condição seja verdadeira e o valor 150 caso contrário. Mantivemos a condição *False* para que a soma seja realizada, conforme desejado no exercício.

7 Questão 7

```
user=> (do  
(println "Iniciando calculo")  
(println "Resultado"(+ 4 6))  
(println "Calculo finalizado"))  
Iniciando calculo  
Resultado 10  
Calculo finalizado  
nil
```

8 Questão 8

O bloco *when* permite executar um bloco de código apenas se uma condição for verdadeira. As instruções são executadas em sequência. O bloco *when* retorna o valor da última linha executada.

Caso a condição seja falsa, a função retornará *nil*. Veja os exemplo a seguir:

```
user=> (when true
(println "Executando bloco when")
(rand-int 10))
Executando bloco when
2
```

```
user=> (when true
(println "Executando bloco when")
(rand-int 10))
Executando bloco when
nil
```

9 Questão 9

```
user=> (def nome-aluno "Arthur")
#'user/nome-aluno
user=> nome-aluno
"Arthur"
user=> (def nota1 8)
#'user/nota1
user=> nota1
8
user=> (str "O aluno "nome-aluno "tirou a nota "nota1)
"O aluno Arthur tirou a nota 8"
```

10 Questão 10

```
user=> (let [a 10 b 5] (println "Soma: "(+ a b))
(println "Produto:"(* a b))
"Os valores de a e b sao locais ao bloco let")
Soma: 15
Produto: 50
"Os valores de a e b sao locais ao bloco let"
```